Praktikum 6: Global Illumination

In der letzten Praktikumsaufgabe geht es darum globale Beleuchtungseffekte einzufügen. Der fleißige Programmierer hat über die Festtage sogar Multithreading eingebaut, damit es jetzt etwas schneller läuft. Bisher wurde die lokale Farbe, die Reflektion und die Brechung deterministisch bestimmt. Für realistische Bilder müssen aber globale Beleuchtungseffekte berücksichtigt werden. Dazu benutzt man Sampling. Wenn ein Strahl auf ein Objekt trifft, bestimmt man die Farbe an diesem Punkt, indem man eine Anzahl von Sampling strahlen in die Szene hineinschießt. Je mehr Samples, umso realistischer der Bildpunkt.

Viel Erfolg!

Bei Fragen oder Problemen schreiben Sie gerne im Moodle!

Aufgabe 1 — Global Illumination

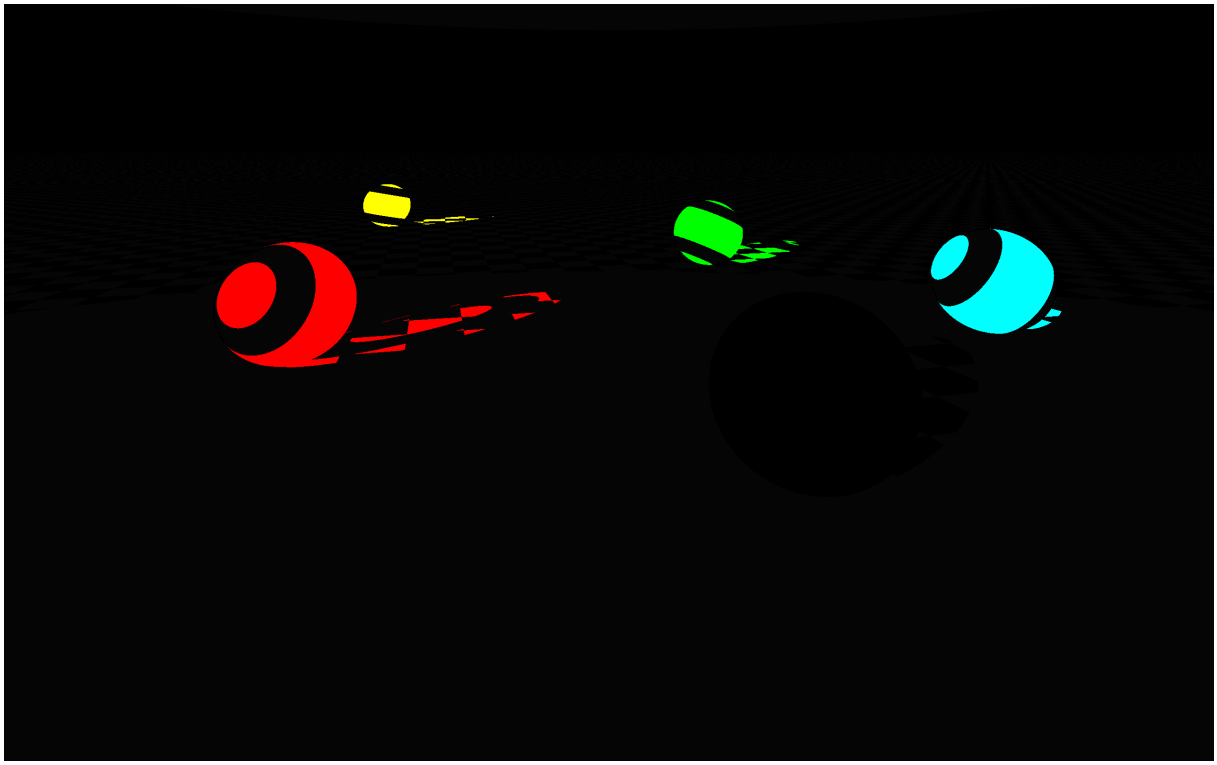


Figure 1: Das hier sollten sie sehen, wenn sie den Code ausführen ohne etwas zu ändern.

a) Öffnen Sie https://git.uni-due.de/vs.ude/objektorientierte-programmierung-cpp. Nutzen Sie git um das Repository zu *clonen* und importieren Sie das Projekt in 06_GlobalIllumination/.

b) In `Scene.cpp` finden Sie die Methode `vec3 get_random_direction`, welche zu einem Normalenvektor (Vektor, welcher immer von einer Oberfläche weg zeigt) einen zufälligen Vektor bestimmt, welcher in die Hemisphäre um den Normalenvektor zeigt. Aktuell ist es ein schlechter Zufallsvektorgenerator, da immer der gleiche Wert zurückgegeben wird.

Ändern Sie den Code sodass ein Vektor mit 3 Zufallswerten die jeweils zwischen –1.0 und 1.0 liegen, zurückgegeben wird.

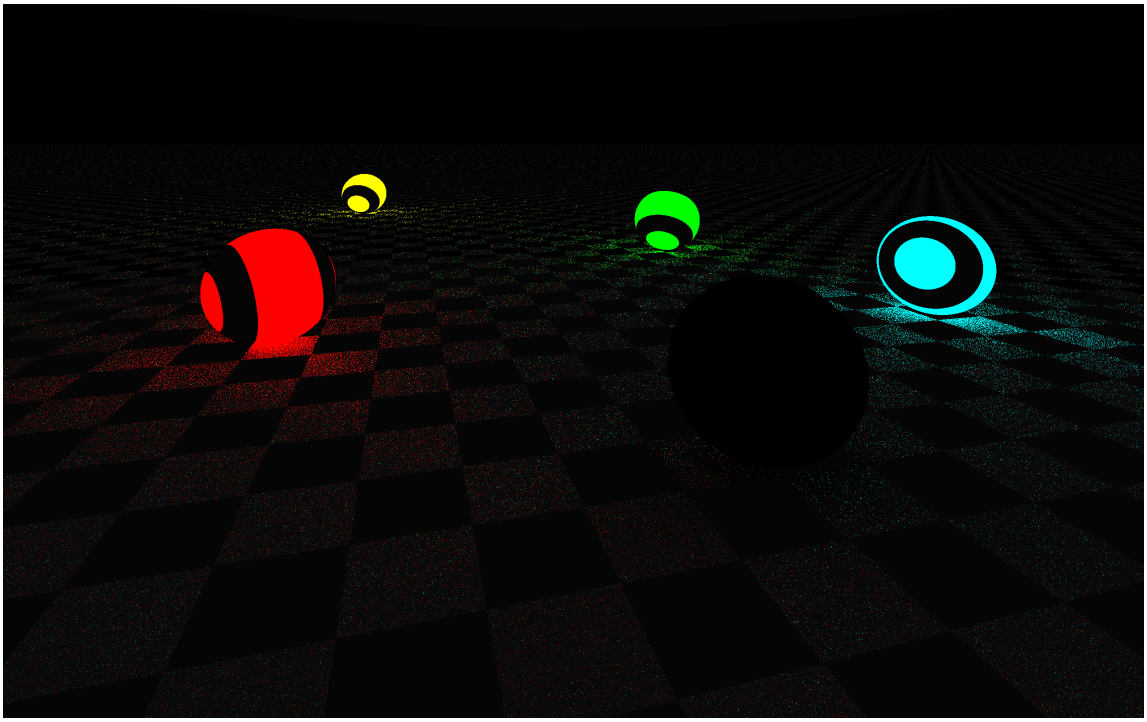


Figure 2: So (oder so ähnlich) sollte es aussehen, wenn das Sampling richtig umgesetzt wurde.

c) In `Scene.cpp` finden Sie die Methode `vec3 Scene::traceRayRealistic`, welche ein realistisches Raytracing implementieren soll. Aufgerufen wird sie parallelisiert in `YourRayTracer::renderRealistic` bzw. `YourRayTracer::render_row`. Schauen Sie sich `Scene::traceRay` an, um sich zu überlegen wie `reflection` berechnet werden wollte, damit die Spiegelung in den Kugeln und der Fläche realistisch aussieht.

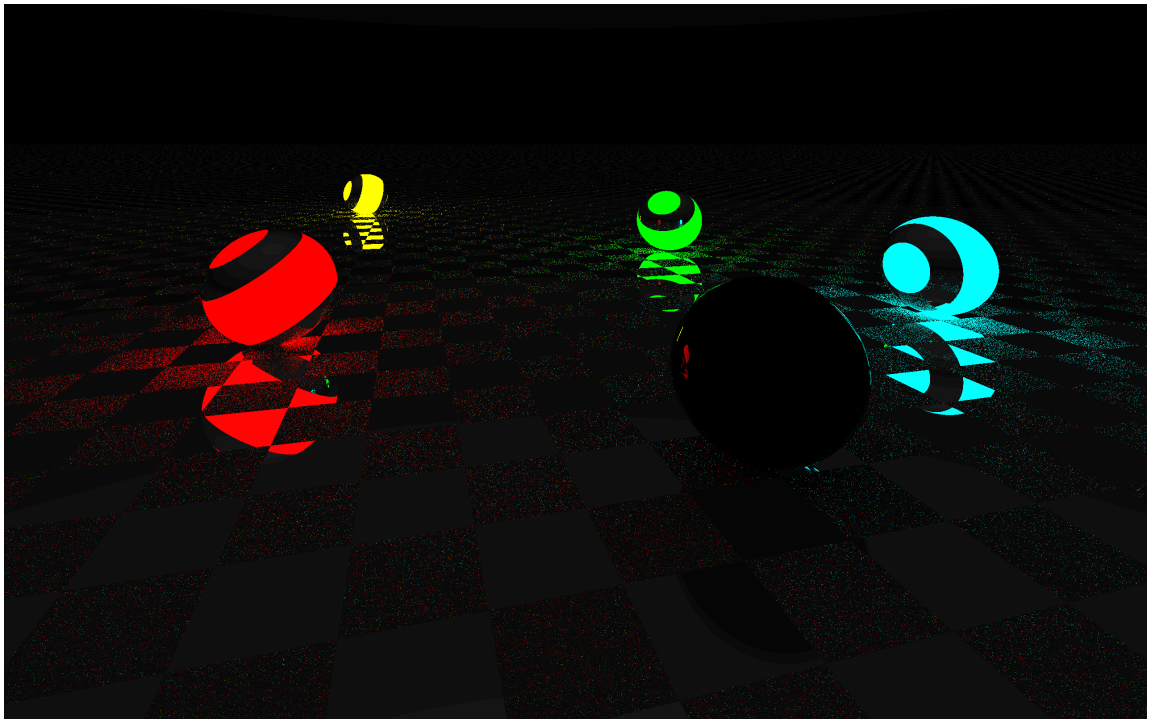


Figure 3: So (oder so ähnlich) sollte es aussehen, wenn Reflektionen richtig umgesetzt wurden.

d) Das selbe für die Brechung. In der Glaskugel vorne können Sie erkennen wie es aussehen sollte.

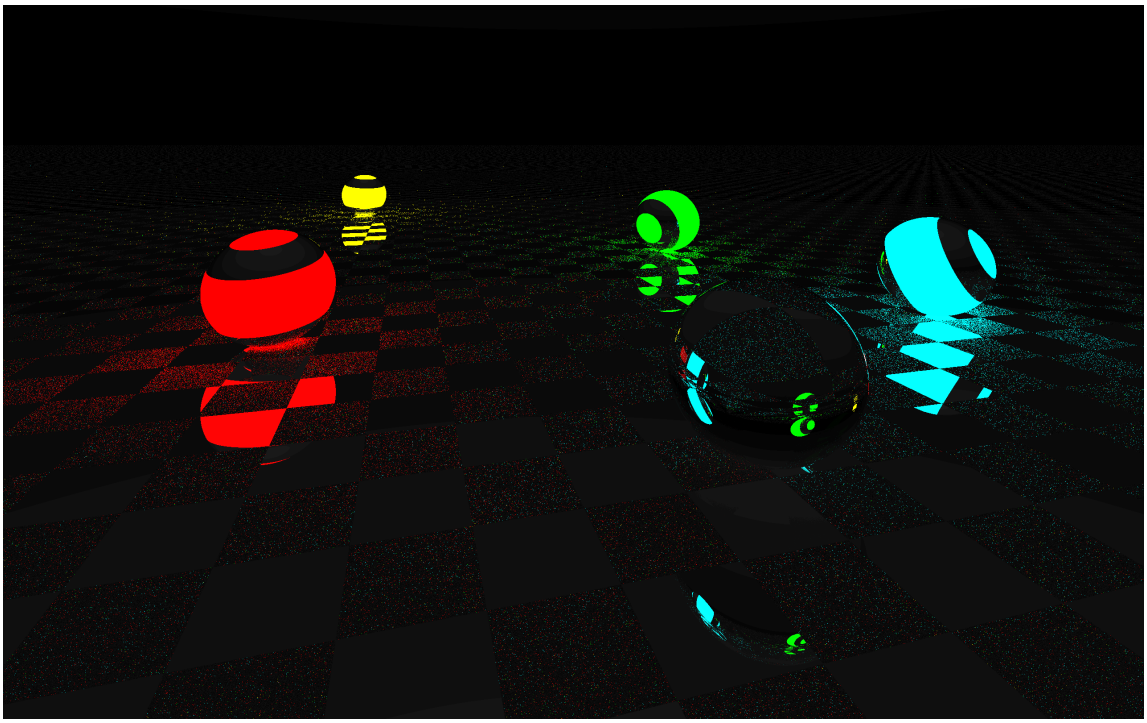


Figure 4: So (oder so ähnlich) sollte es aussehen, wenn Brechungen richtig umgesetzt wurden. Bei wenigen Samples

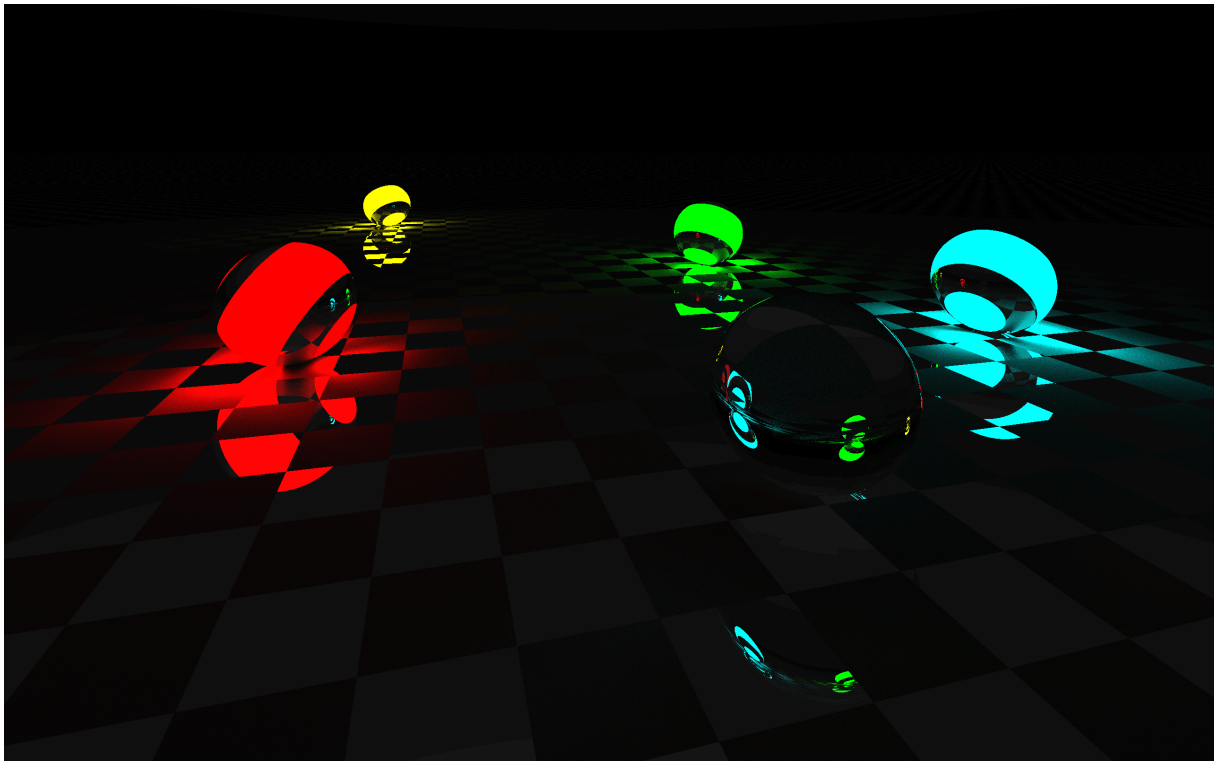


Figure 5: In `main.cpp` können Sie bei der Erstellung der `Scene` einstellen, wie viele Samples genutzt werden sollen. Bei 1000 Samples kann es dann wie folgt aussehen.

Falls Sie es bis hierhin geschafft haben, herzlichen Glückwunsch! Ich hoffe es hat Ihnen Spaß gemacht dieses Praktikum zu bestreiten und Sie konnten etwas für ihre Weiterbildung mitnehmen. Computergrafiken sind ein riesiges Feld und es gibt zahllose Optimierungen die noch möglich wären, damit die Bilder noch schöner oder realistischer aussehen. Ich wünsche Ihnen viel Erfolg bei der Vorbereitung für die Klausur.